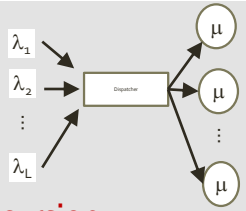
FVRL: Fleming-Viot Reinforcement Learning
*A method for **faster optimization***

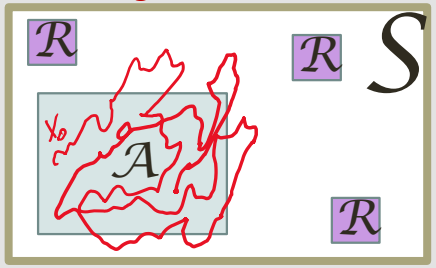
Applications to queueing networks

***Daniel Mastropietro**, Urtzi Ayesta, Matthieu Jonckheere, Symon Majewski*

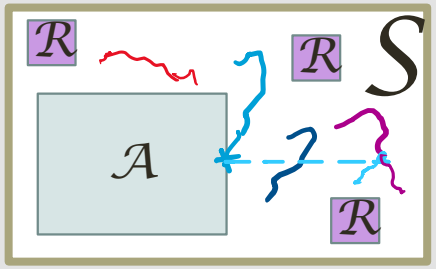
PyData Lausanne, 26-Nov-2025

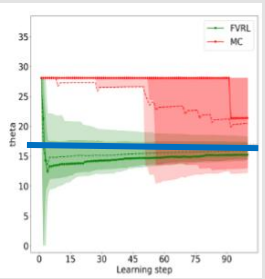


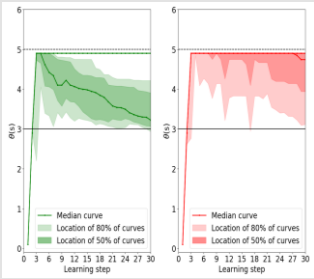
Regular excursion



Fleming-Viot excursion





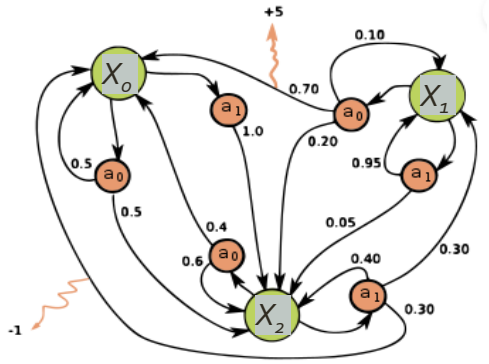


RÉPUBLIQUE FRANÇAISE
TOULOUSE INP
IRIT Institut de Recherche en Informatique de Toulouse CNRS - INP - UT3 - UT1 - UT2J
LAAS CNRS

Reinforcement Learning (RL) is an optimisation approach that requires very few modelling assumptions.

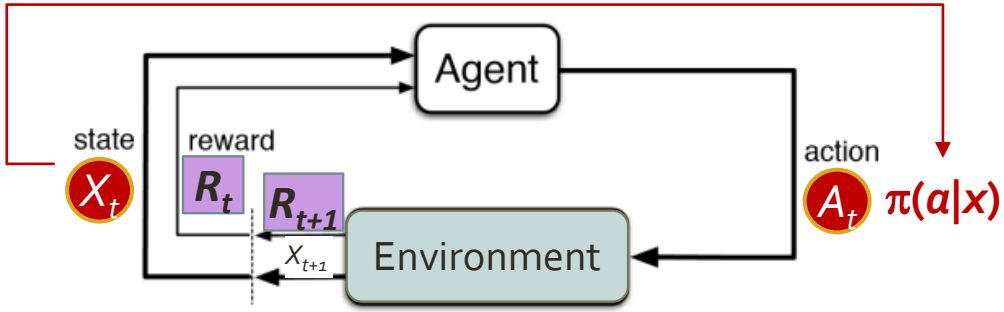
- **Context:**
  Optimisation of long-term objectives in Markovian systems.
- **What is RL?**
  Learning from interactions between an agent and the system.
- **In practice:**
  Optimize by "trial and error"

  Environment exploration — Signal returned (*reward*)
- **What is optimized?**
  Actions / Decisions at *each* state *x* of the system, i.e. the Policy: $\pi(a|x)$
- **With what goal?**
  Maximize / Minimize a long-term objective.
  Ex: the long-run expected reward.



Source: Wikipedia

Markov Decision Process (MDP)



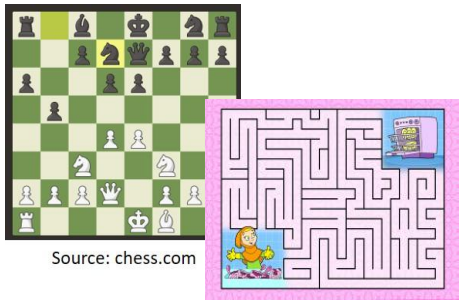
The **data** for learning are trajectories:

$X_0 \rightarrow A_0 \rightarrow R_1 \rightarrow X_1 \rightarrow A_1 \rightarrow R_2 \rightarrow \ldots$

Defined stochastically by policy $\pi(a|x)$

Note: Depending on the problem, rewards may be actually "costs".

RL Examples

	Games	Business
Problem	 Source: chess.com	 
Long-term objective	**Win** the game.	**Quality of Service** with limited resources.
States (X_t)	Board **configuration.** Player **position**.	**No. jobs** in service. Energy **demand**.
Actions / Decisions (A_t)	**Move** a piece. **Move** the player.	**Accept/Reject** a job. **Deliver/Cut** distribution.
Rewards (R_t)	**Value of piece** taken. **Prize** is found.	Cost of **rejection**. Cost of **black-out**.

Advantages and Disadvantages of RL

ADVANTAGES

Little model requirements

(*Markovian assumption is enough*)

Complex **optimisation problems**

Online **learning**

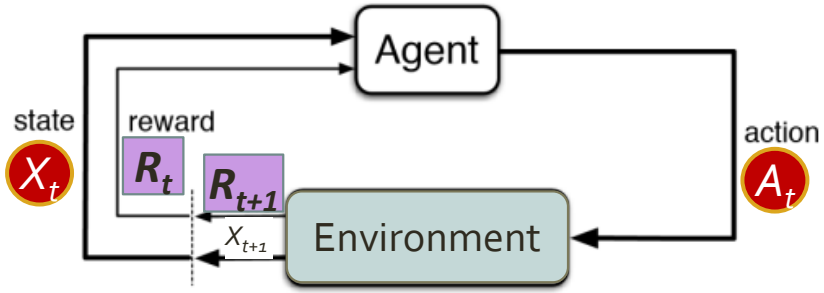
Can handle *non-stationarity*

DISADVANTAGES

Time and resource consuming

(*Exploration of all the environment*)

Balance ***exploration*** *(of environment) vs.* ***exploitation*** *(of signal)*



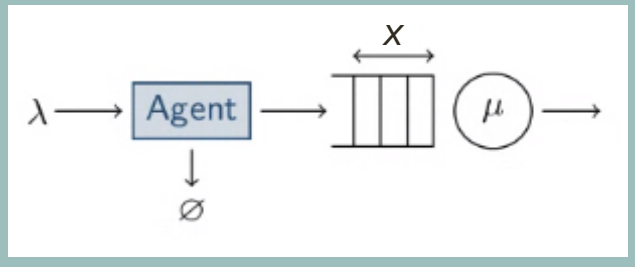
Our goal:

Reduce time and resource consumption by ***increasing exploration efficiency*** *so that the signal can be* ***exploited*** *faster.*

Motivating example

A simple queueing system where rewards are ***rare***

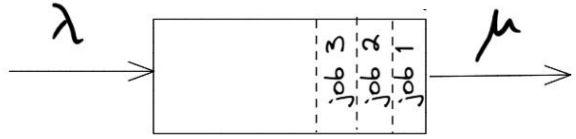
Queue system dimensioning



M/M/1 queue

λ: job arrival rate
(exponential inter-arrival times)

μ: service rate
(exponentially distributed times)



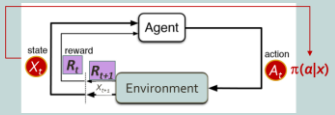
X(t) = "Number of jobs in the queue"

Given that:

- *There are NO job holding costs.*
- *There is a **rejection cost** of an incoming job that **increases with the queue size X(t)** at the time of rejection.*

Goal:

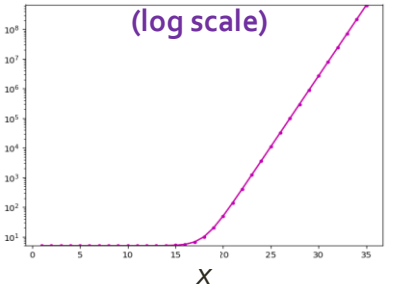
*Design a threshold-type job admission control policy **π(a|x)** that **minimizes the average cost in the long-run**.*



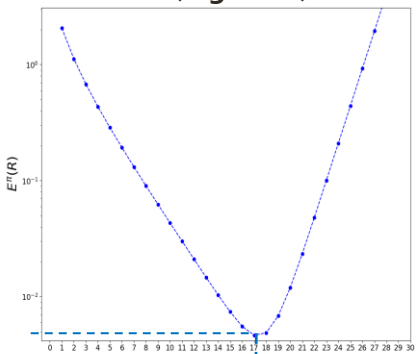
Notes:

- *Two possible actions **a**: Accept or Reject an incoming job.*
- *The Acceptance / Rejection threshold defines how many jobs the queue can hold (its capacity **K**) => **The goal is to optimize K**.*

Rejection cost as a function of x (log scale)



Average cost in the long-run (log scale)

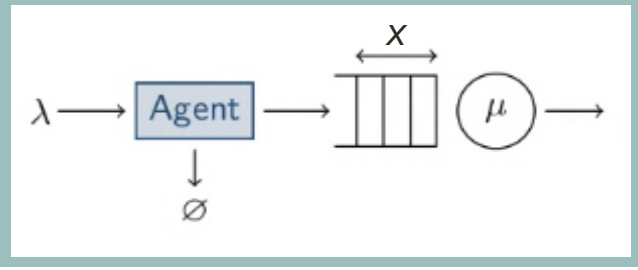


Optimum K

Motivating example

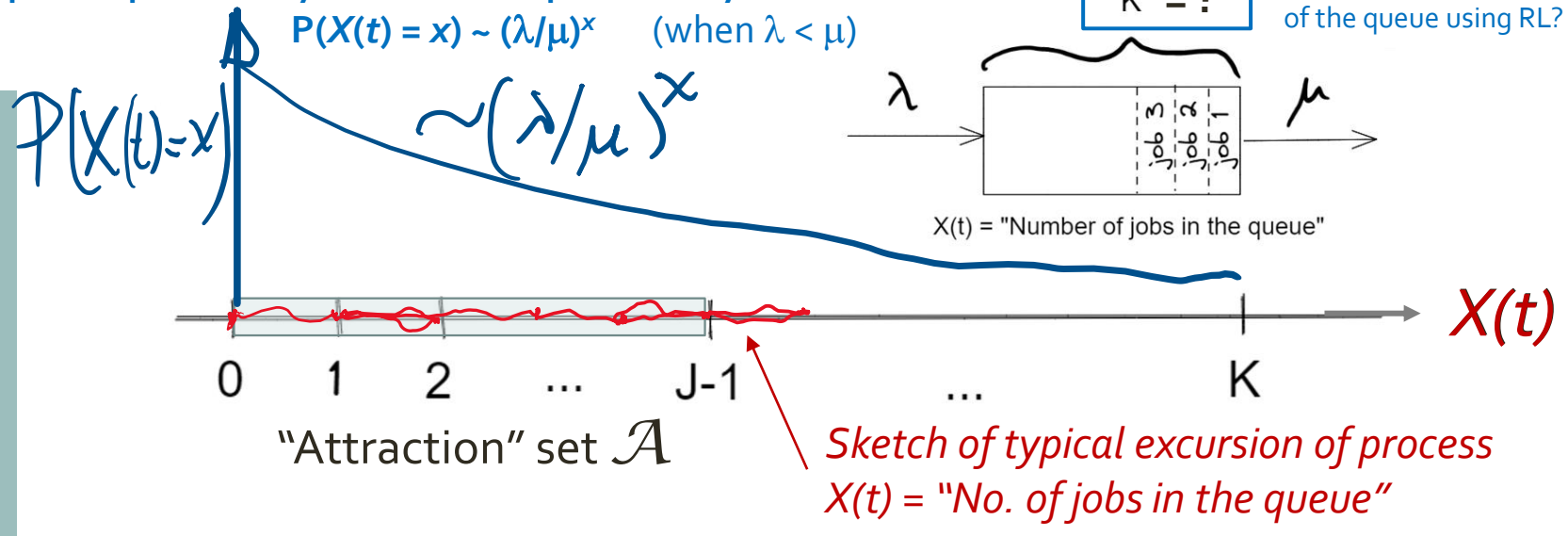
A simple queueing system where rewards are ***rare***

Queue system dimensioning



Stationary occupation probability decreases exponentially with x:

$P(X(t) = x) \sim (\lambda/\mu)^x$ (when $\lambda < \mu$)



- The states in $\mathcal{A}$ are frequently visited BUT give no cost information (*zero reward*)
- Blocking is **rare** when K is large or $\lambda \ll \mu$
  e.g. large K: $K = 40,\ \lambda = 0.7, \mu = 1 \implies Pr(X(t) = K) \sim 10^{-7}$

If no rejection (queue blocking) is observed…
the average cost will be estimated as **ZERO**

It will be difficult to optimize *K*
to minimize the average blocking cost

(Summing up)

Our goal

Develop an RL algorithm for **sample-efficient solution** of stochastic **optimisation** problems

(Summing up)

The framework

Optimize a
long-term objective (average cost)
whose value depends on
decisions taken over time (accept/reject job)
potentially affected by
large rare rewards (rejection cost)

Mathematical framework:
Markov decision processes (MDP)

Methodological framework:
Reinforcement learning (RL)

Optimisation output:
Policy π = Probability(action | state)

(Summing up)

Why Reinforcement Learning (RL)?

RL is **model-free**
and can find optimal policies
by **trial and error**

It's **appealing** for solving
complex optimisation problems
(difficult to model or highly nonlinear)

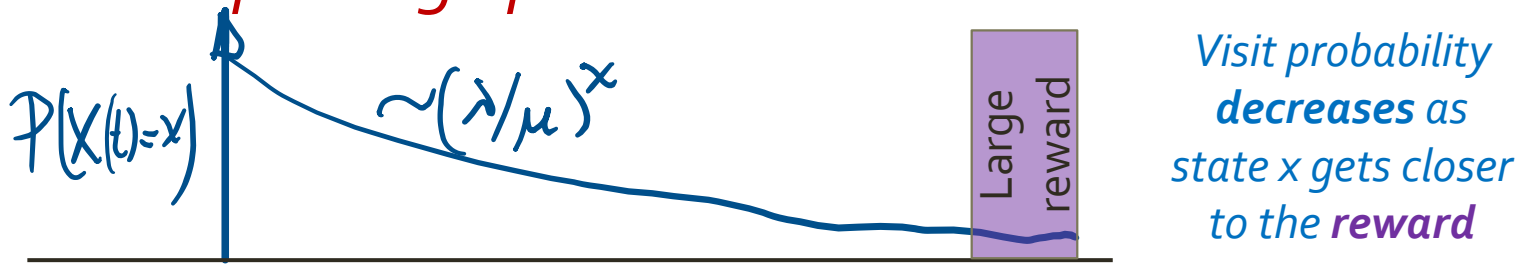
(Summing up)

but there are **challenges**

How long it takes
to find an optimum?

Computing intensive → *How to **discover** the environment **efficiently in large configuration spaces**?*

Uneven probability distribution → *A few states could be **rarely visited** but yield **very large rewards** impacting optimisation…*



Efficient exploration (to find **rewards**) is one of the main problems in RL

Existing methods

METHOD	DESCRIPTION	CHALLENGE
Importance Sampling	*Alternative exploration policy* to boost visit of rare events.	*Not always possible* to apply. (ex. Queue)
Reward shaping	*Engineer rewards* to help learn faster.	Requires *domain knowledge*.
Curiosity-driven methods	*Bonus* in objective function to encourage exploring elsewhere.	*Bonus choice* may be challenging for good balance between exploration vs. exploitation.

Our proposal: FVRL (Fleming-Viot RL)

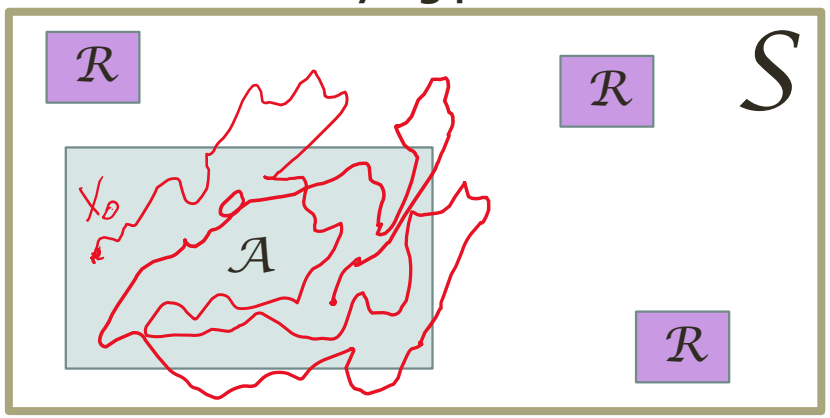
METHOD	DESCRIPTION	CHALLENGE
FVRL (Fleming-Viot RL)	*Curiosity-driven method*, where bonus is replaced with ***forcing*** exploration away from frequently visited states.	Requires a *simulator.*

Main idea

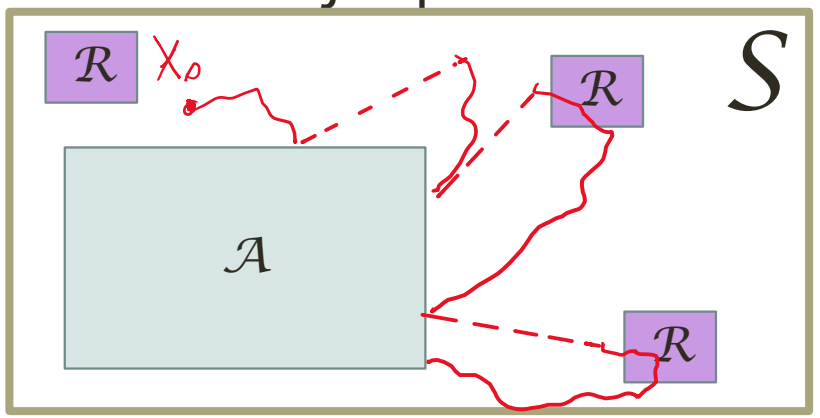
Our proposal: FVRL (Fleming-Viot RL)

Leverage information about frequently visited states to force exploration elsewhere

Underlying process



***Modified* process**



S Set of states

A trajectory of the process, mostly visiting "atraction" states

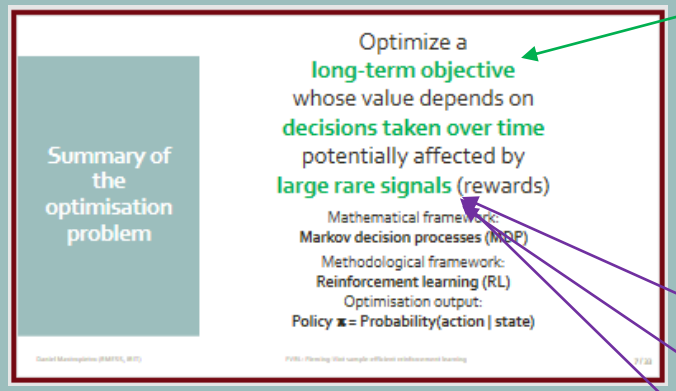
$\mathcal{A}$ Set of "*Atraction*" states: frequently visited & possibly zero-reward states

$\mathcal{R}$ Set with rarely observed states with potentially large rewards

$\mathcal{A}$ known or easily learned $\mathcal{R}$ unknown

A trajectory starts near $\mathcal{A}$ and is forced outside.

- The process stays away from the set of "atraction" states $\mathcal{A}$
- Rare states $\mathcal{R}$ are observed more frequently => *the process has higher* ***sample-efficiency*** *for discovering* $\mathcal{R}$ *states.*



Our proposal

Efficiently estimate the stationary state distribution $p^{\pi}(x)$

We consider problems where the **long-term objective** is the **long-run expected reward**

$$E^{\pi}(R) = \sum_{x} p^{\pi}(x) r(x)$$ unknown / known

$p^{\pi}(x)$: long-run (stationary) state distribution.

$r(x)$: reward function of the state.

Recall the goal of the optimisation problem

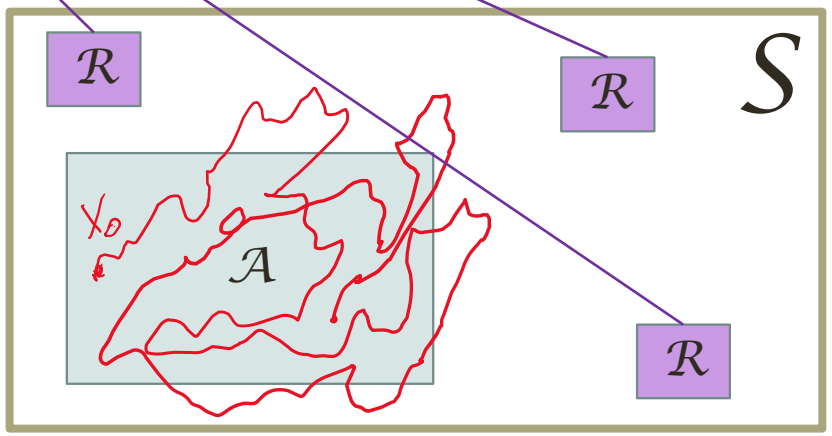
Find an optimal policy π = **Probability(action | state)**

To optimize we need a way to measure the objective function

=> We need to estimate $p^{\pi}(x)$ for each possible policy π

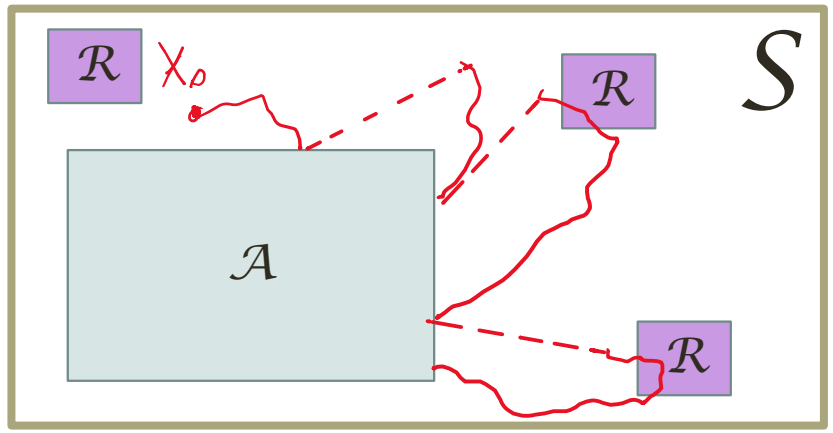
We propose a sample-efficient estimator of $p^{\pi}(x)$.

Our method replaces this...



Non-sample-efficient discovery of $\mathcal{R}$ states

...with this.



Sample-efficient discovery of $\mathcal{R}$ states

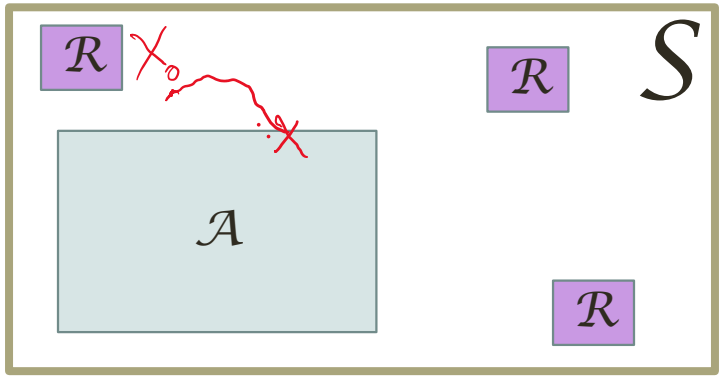
Remark: $\mathcal{A}$ stands for "Atraction" and "*Absorption*"

→ We leverage a **stochastic process with *absorption*** to construct our estimator of $p^{\pi}(x)$

Fleming-Viot particle system *helps us estimate* $p^{\pi}(x)$ *efficiently*

Stochastic process with absorption

*A stochastic process in which the **process stops** when reaching a certain condition*

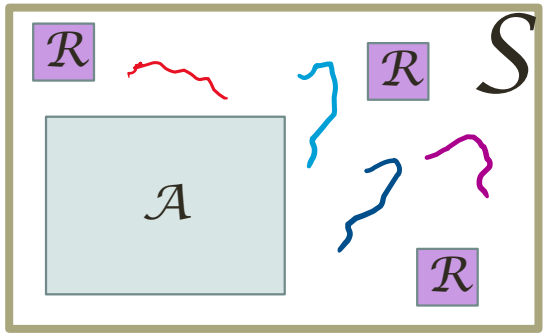


Example: process **X(t) stops** when reaching a state in A.

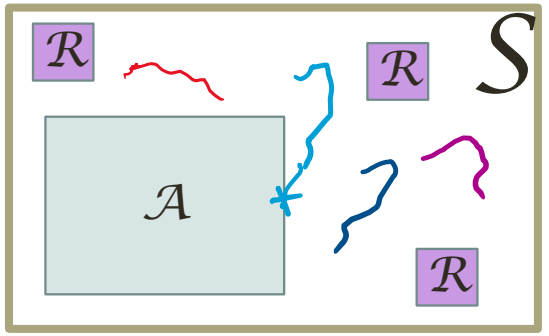
Fleming-Viot particle system (1979)

Stochastic process constructed on top of a ***stochastic process with absorption*** *to model population evolution.*

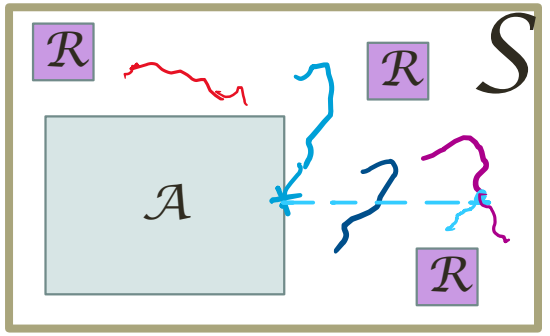
1. N particles evolve in **parallel** *under same policy*



2. At some point **a particle is *absorbed***

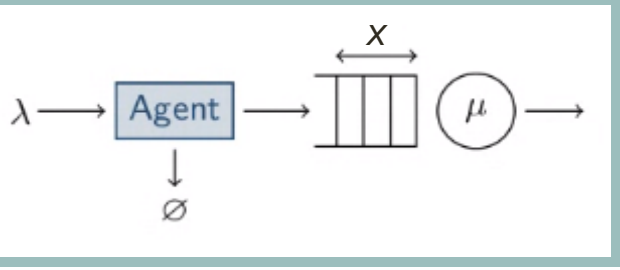


3. Absorbed particle is **immediately regenerated**



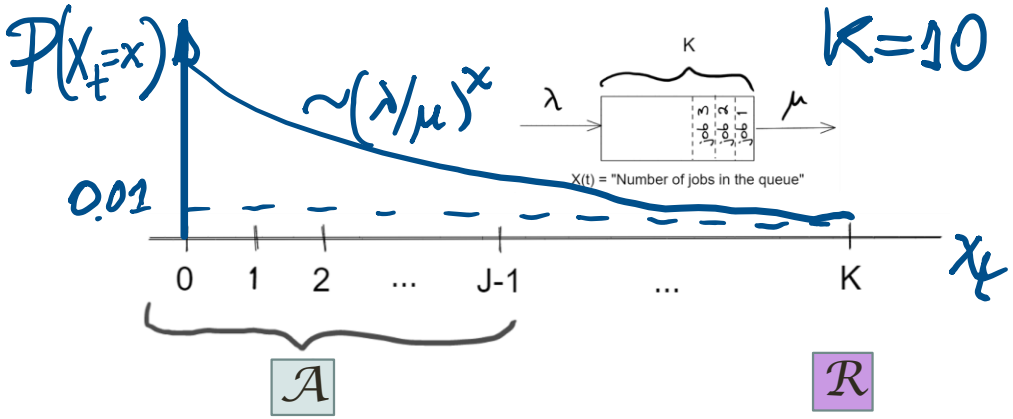
Example: M/M/1/K queue

Fleming-Viot process stays closer to the rare reward than original process

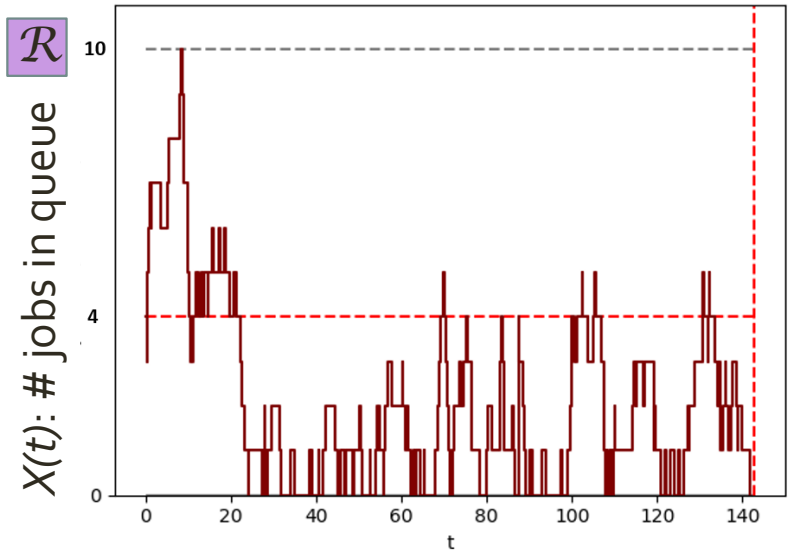


M/M/1/K queue characteristics:

- Service load: $\lambda/\mu = 0.7 < 1$
- Max queue size: $K = 10$
- $\Pr(X(t) = K) \sim 0.01$

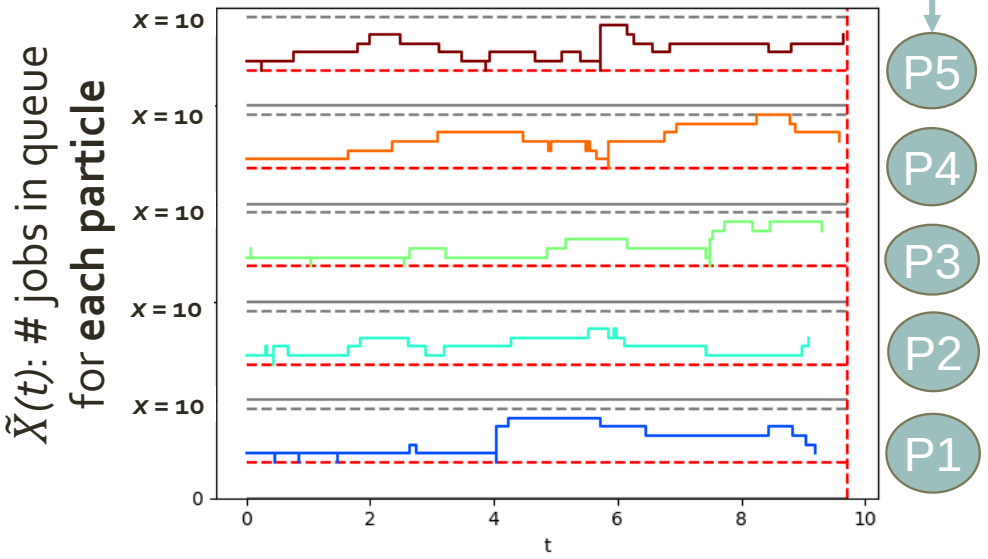


One realization of underlying process (the MDP)



Fleming-Viot process with J-1=4

$\mathcal{A} = \{0, 1, \ldots, 4\}$ on N=5 particles



While the **number of Markov process transitions is the same** in both cases, the **Fleming-Viot process** tends to be **closer to the rare state *K*** with reward $\mathcal{R}$

Fleming-Viot estimation of $p^{\pi}(x)$ is based on renewal theory

Recall our objective…

We consider problems where the **long-term objective** is the **long-run expected reward**

$$E^{\pi}(R) = \sum_{x} p^{\pi}(x) r(x)$$ **unknown / known**

$p^{\pi}(x)$: long-run (stationary) state distribution.
$r(x)$: reward function of the state.

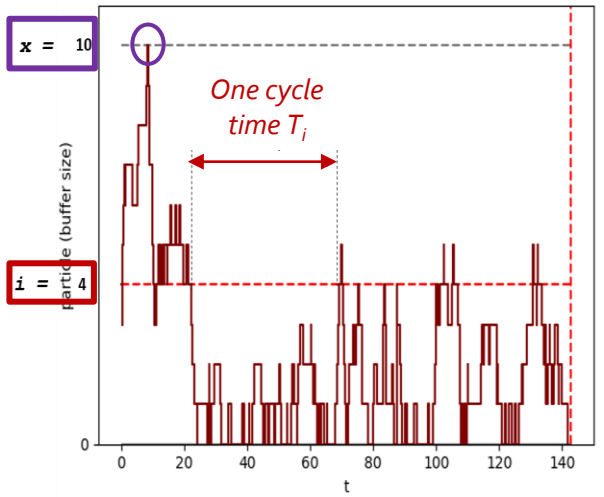
Renewal theory uses cycles (interval between two consecutive occurrences of the same event) to estimate the stationary distribution $p^{\pi}(x)$.

M/M/1/K example: $E^{\pi}(R) = p^{\pi}(K) r(K)$ as r(x) = 0, for $x \neq K$

We need $p^{\pi}(x)$ in state $x = K = 10$

We choose **event "i": "visit $x = 4$"**

We call this **Monte-Carlo** estimation of the the stationary distribution $p^{\pi}(x)$



Renewal theorem

$$p^{\pi}(x) = \frac{E_i^{\pi} \int_0^{T_i} I\{X(t) = x\}\, dt}{E_i^{\pi}(T_i)} = \frac{\text{Average time spent in state } x}{\text{Average return time to } i}$$

We now apply the renewal theorem on the Fleming-Viot particle system to estimate $p^{\pi}(x)$

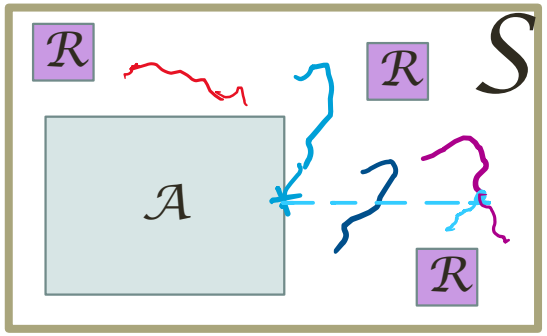
Recall our objective…

We consider problems where the **long-term objective** is the **long-run expected reward**

$$E^{\pi}(R) = \sum_{x} p^{\pi}(x) r(x)$$ **unknown / known**

$p^{\pi}(x)$: long-run (stationary) state distribution.
$r(x)$: reward function of the state.

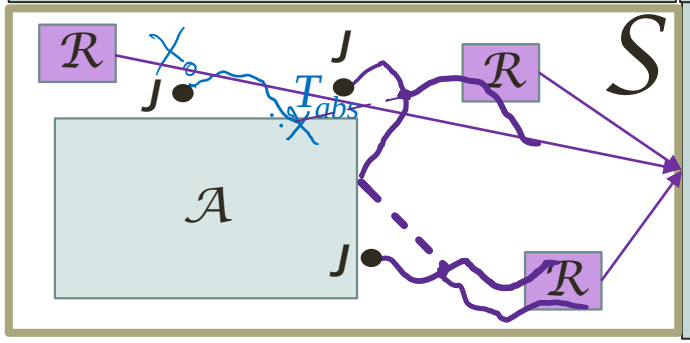
Recall the **Fleming-Viot particle system…**



We *define* the **event defining a cycle**: **"Reabsorption to $\mathcal{A}$"** (→ *i = J-1*)

Can be expressed as: $p^{\pi}(x)$

$$= \frac{\int_0^{\infty} P_J^{\pi}(T_{abs} > t)\, P_J^{\pi}(X(t) = x | T_{abs} > t)\, dt}{\text{Expected return time to } J\text{–}1 \text{ under } \pi}$$



$P_J^{\pi}(T_{abs} > t)$: survival distribution of "**Time until absorption**";

$P_J^{\pi}(X(t) = x | T_{abs} > t)$: state distribution while not absorbed;

when process starts at a state (*J*) just outside $\mathcal{A}$.

Fleming-Viot estimator of $p^{\pi}(x)$ requires the estimation of 3 quantities

For states x outside $\mathcal{A}$

$$p^{\pi}(x) = \frac{\int_0^{\infty} P_J^{\pi}(T_{abs} > t) P_J^{\pi}(X(t) = x | T_{abs} > t)\, dt}{\text{Expected return time to } J-1 \text{ under } \pi}$$

Need to **ESTIMATE**:

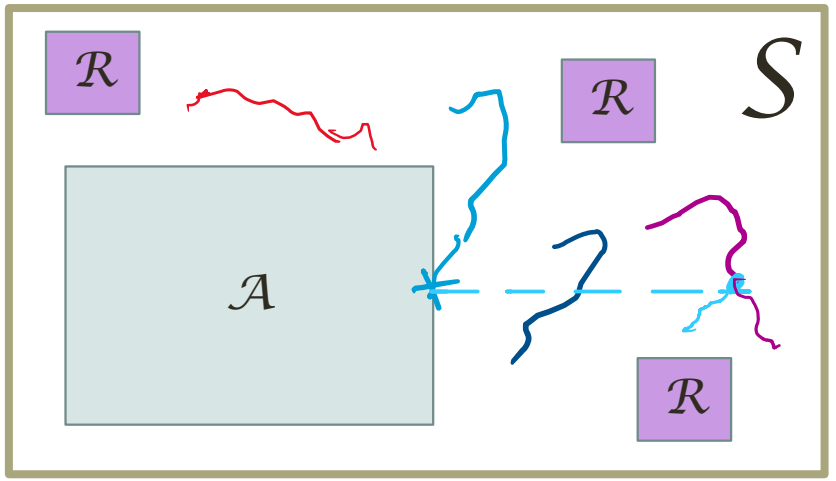
1) Expected return time to $J-1$ under π (moment estimator)
2) $P_J^{\pi}(T_{abs} > t)$: Survival distribution of "Time until absorption" (moment estimator)
3) $P_J^{\pi}(X(t) = x | T_{abs} > t)$: State distribution while not absorbed. (Fleming-Viot particle system)

The **implementation is more complex** than the usual uninformed estimation of $p^{\pi}(x)$ but is able to reach rarely visited states faster…

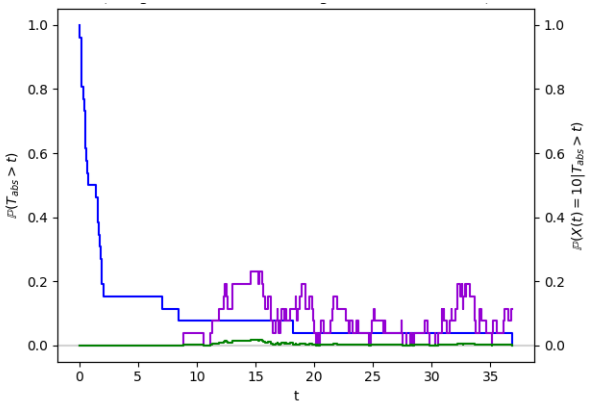
… and it pays off in terms of **sample efficiency**.

IMPORTANT: Trajectories are obtained from SIMULATION of the system

The Fleming-Viot particle system is used to estimate $\boldsymbol{P_J^{\pi}(X(t) = x | T_{abs} > t)}$ *(the state distribution conditioned to non-absorption)*



Typical estimators (with N = 30 particles) of $\boldsymbol{P_J^{\pi}(T_{abs} > t)}$, $\boldsymbol{P_J^{\pi}(X(t) = 10 | T_{abs} > t)}$ and their product.



The estimation of $\boldsymbol{P_J^{\pi}(X(t) = x | T_{abs} > t)}$ (for each *t*) is surprisingly simple.

Let $m_{N,t}(x)$ = "proportion of *N* particles in state *x* at time *t*", then:

$\boldsymbol{m_{N,t}(x) \to P_J^{\pi}(X(t) = x | T_{abs} > t)}$ as $N \to \infty$ with error ~ $O(1/\sqrt{N})$

Asselah, A.; Ferrari, P. A.; Groisman, P. (2011)
"*Quasistationary Distributions and Fleming-Viot Processes in Finite Spaces*"

NOTE: In the general **multidimensional-state space**:
Starting state *J* becomes "*the outside boundary of attraction set* $\mathcal{A}$".

The Fleming-Viot estimator of $p^{\pi}(x)$ is consistent

$$p^{\pi}(x) = \frac{\int_0^{\infty} P_J^{\pi}(T_{abs} > t) P_J^{\pi}(X(t) = x | T_{abs} > t)\, dt}{\text{Expected return time to } J\text{–}1 \text{ under } \pi}$$

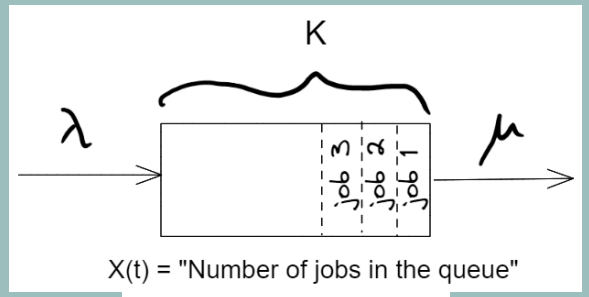
The expected absolute error is bounded:

$$E\left|\hat{p}_{FV}^{\pi}(x) - p^{\pi}(x)\right| \leq C\left(\frac{1}{\sqrt{N}} + \frac{1}{\sqrt{M}}\right)$$

where:

- N is the number of particles in the FV particle system
- M is the number of return cycles to $\mathcal{A}$ observed by the ONE-particle system simulation estimating the denominator

D. Mastropietro, S. Majewski, U. Ayesta, M. Jonckheere (2022)
"*Boosting reinforcement learning with sparse and rare rewards using Fleming-Viot particle systems*"
https://ut3-toulouseinp.hal.science/hal-03772025



λ = 0.7, μ = 1

Experimental results:

Fleming-Viot vs. Monte-Carlo estimation

K = 20
True $p(K)$ = **2.4E-4**

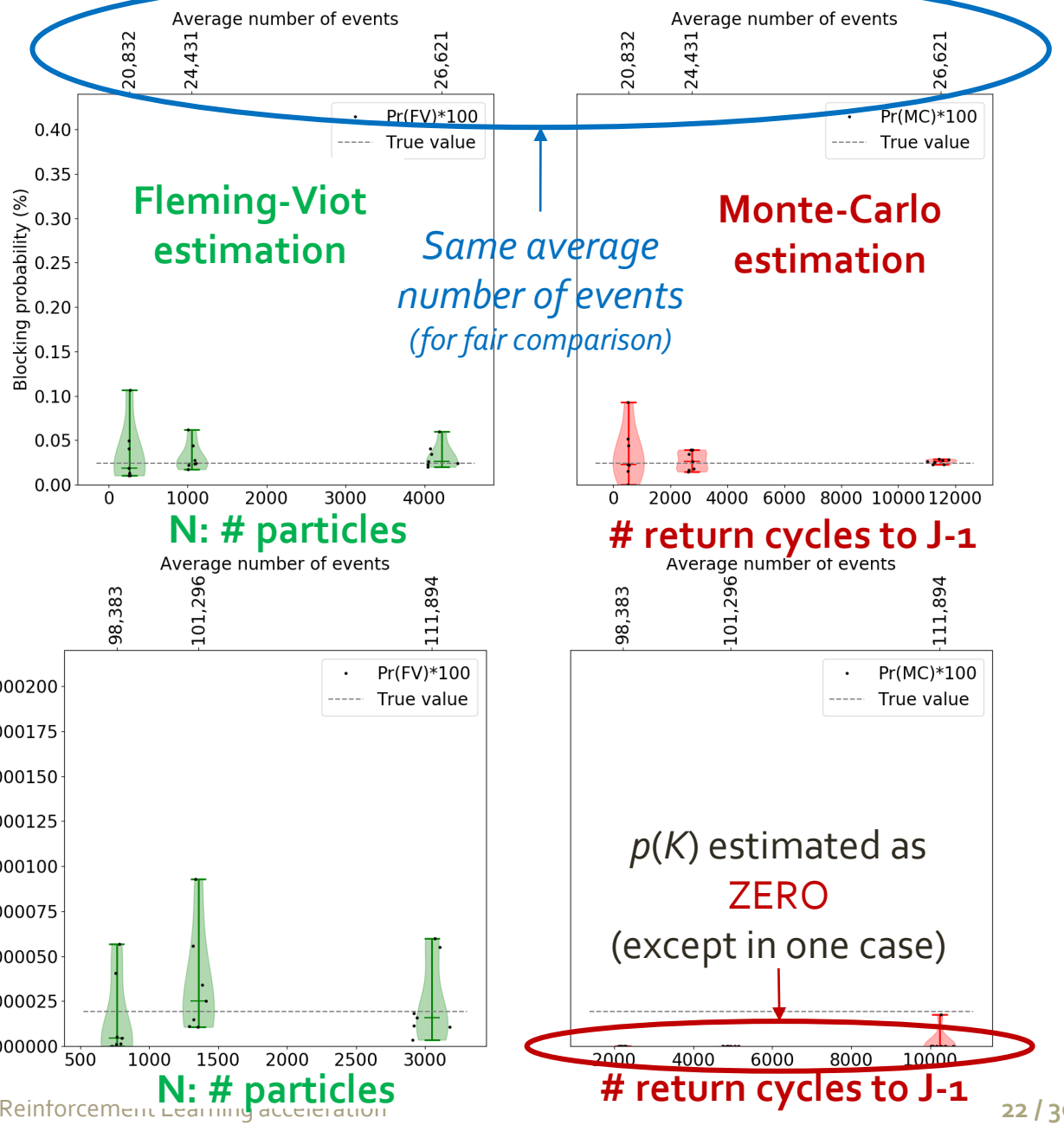
FV hyperparameters:

- **N: # particles increases** (horizontal axis)
- T = 4000 arrivals
- J = 12
- 10 replications per N

K = 40
True $p(K)$ = 1.9E-7

FV hyperparameters:
(same as above)

Closer to horizontal dotted gray line and shorter violin plots is better



FVRL: Fleming-Viot Reinforcement Learning

Fleming-Viot estimation combined with policy gradient

FVRL: Learn an optimal policy π that maximizes the long-run expected reward,

$$E^{\pi}(R) = \sum_{x} p^{\pi}(x) r(x)$$

using the **Fleming-Viot estimation** of the stationary distribution $p^{\pi}(x)$.

The policy is parameterised so that we can apply a gradient ascent algorithm:

$$\pi \doteq \pi_{\theta}$$

where θ is a d-dimensional real vector.

An optimal policy is one that satisfies:

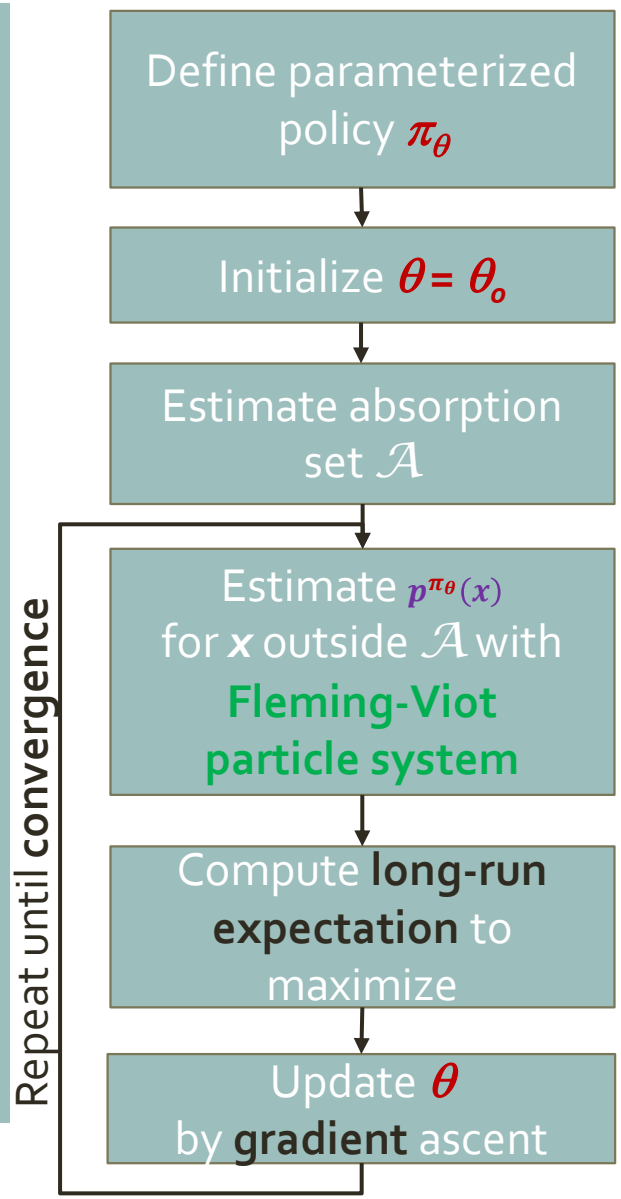
$$\pi^{*} \doteq \pi_{\theta^{*}} = \underset{\theta \in \mathbb{R}^{d}}{\operatorname{argmax}} E^{\pi_{\theta}}(R)$$

It is learned by gradient ascent, using the policy gradient theorem(*), by which:

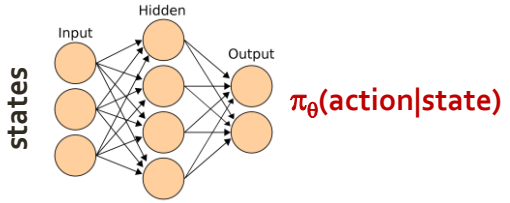
$$\nabla E^{\pi_{\theta}}(R) \sim \nabla \pi_{\theta}$$

(*) E.g. see "Reinforcement Learning", Sutton & Barto (2018), Ch. 13

FVRL workflow



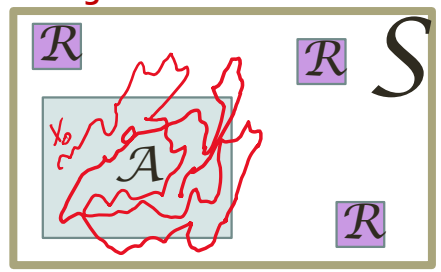
E.g. **Neural network** model



E.g. **Random walk** policy

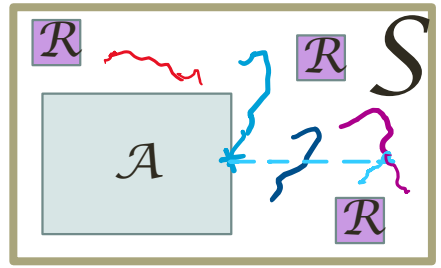
E.g. Based on initial excursion and **visit frequency threshold**

Regular excursion of MDP



$$p^{\pi_\theta}(x) = \frac{\int_0^\infty P_J^{\pi_\theta}(T_{abs} > t) P_J^{\pi_\theta}(X(t) = x | T_{abs} > t)\, dt}{\text{Expected cycle time to } J\text{–}1 \text{ under } \pi_\theta}$$

Fleming-Viot particle system



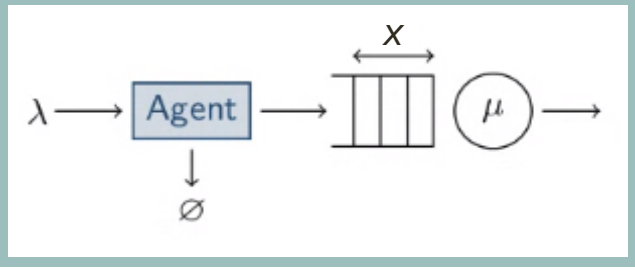
$$E^{\pi_\theta}(R) = \sum_x p^{\pi_\theta}(x) r(x)$$

$$\nabla E^{\pi_\theta}(R) \sim \nabla \pi_\theta$$

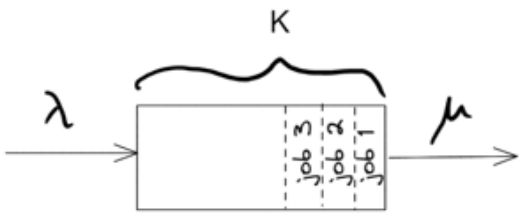
- Fleming-Viot estimator of $p^{\pi_\theta}(x)$ is biased but **consistent**.
- Convergence guarantees of FVRL to **local optimum**.

Results (1): M/M/1/K queue

Objective: find the optimal capacity *K*



M/M/1/K system

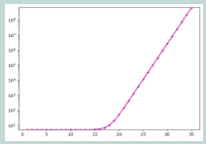


Service load: $\lambda/\mu = 0.7$

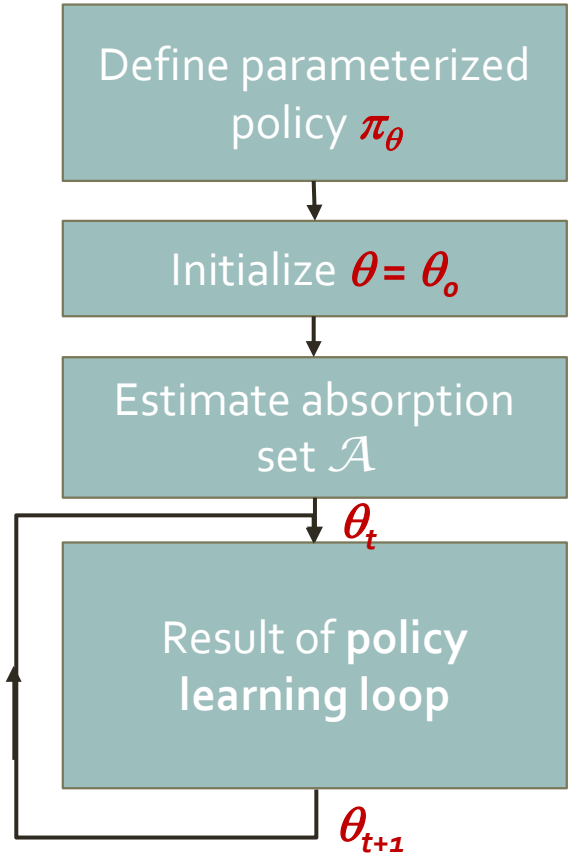
Rejection cost: r(*K*) ~ -b^{K} (some b > μ/λ)

Possible actions:
Accept or Reject an incoming job

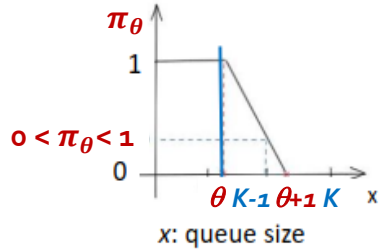
Reward landscape, r(x):
All ZERO, except at job rejection, with *exponential cost*



Function to maximize:
Long-run expected reward: $E^{\pi}(R) = p^{\pi}(K)r(K)$



Acceptance probability is
step function of $\theta \in \mathcal{R}$
=> *Policy gradient ≠0, only at $\theta = K-1$*



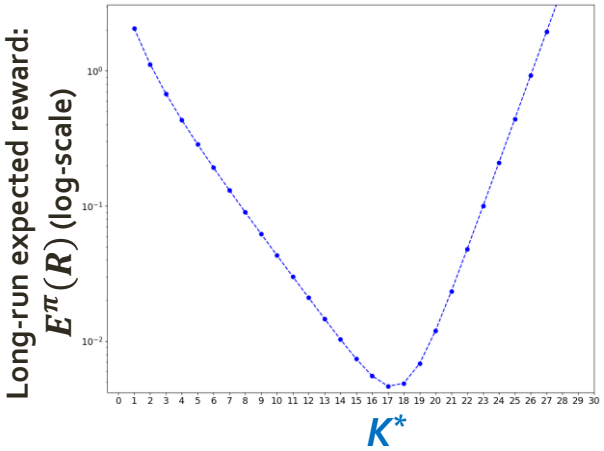
Initial policy:
Reject at large *K* = 28

Probability(*K*) = 10^{-5}

States with
visit frequency > 0.5%

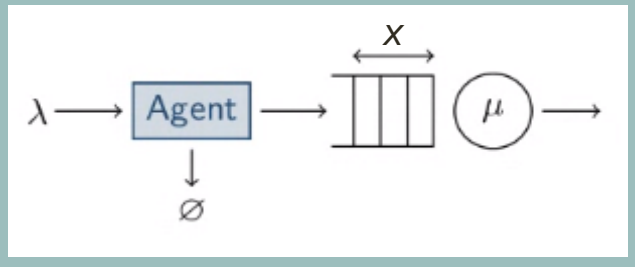
Size of absorption set $\mathcal{A}$:
~ *K*/3

Function to optimize

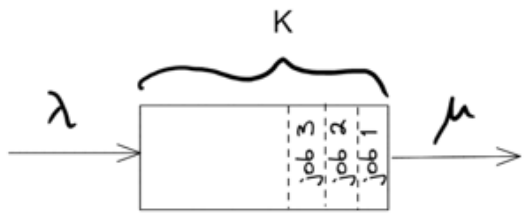


Results (1): M/M/1/K queue

Objective: find the optimal capacity *K*



M/M/1/K system

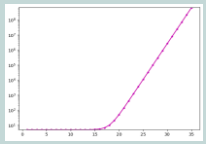


Service load: $\lambda/\mu = 0.7$

Rejection cost: $r(K) \sim -b^K$ (some $b > \mu/\lambda$)

Possible actions:
Accept or Reject an incoming job

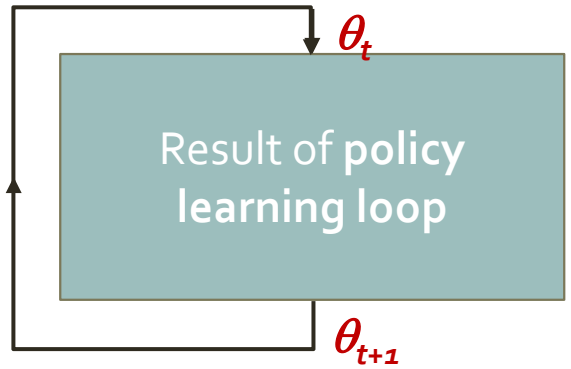
Reward landscape, r(x):
All ZERO, except at job rejection, with *exponential cost*



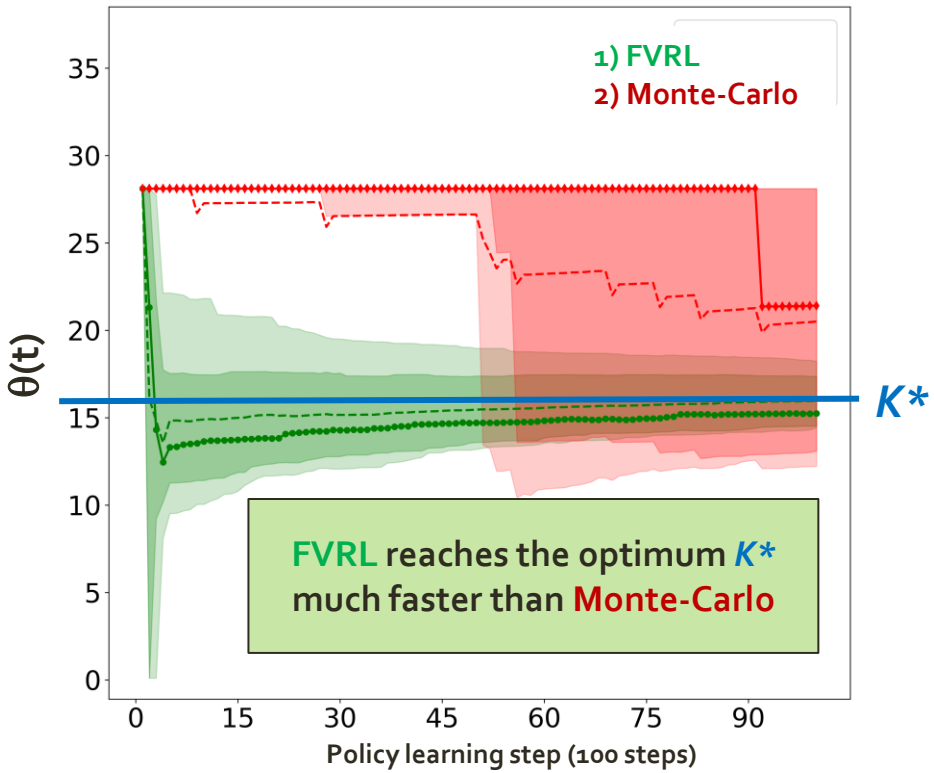
Function to maximize:
Long-run expected reward: $E^{\pi}(R) = p^{\pi}(K)r(K)$

FVRL setup:
- # particles < 100: smaller as K becomes smaller
- # queue transitions per policy learning step ~ 4,000

Monte-Carlo setup:
- Same number of queue transitions as Fleming-Viot per policy learning step

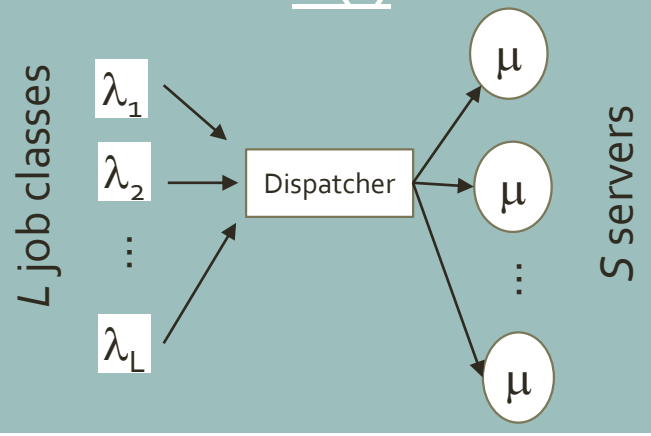


Long-run expected reward: $E^{\pi}(R)$
(20 replications)



Results (2): **Loss Network**

Objective: find optimal capacities $K(l)$



M/M/L/S system

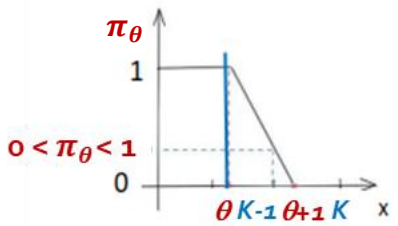
L = 2 job classes
S = 6 servers

Rejection cost depends on the job class: *R(l)*

One policy *π(l)* for *each* job class

Multidimensional problem: $\theta \in \Re^2$

Acceptance probability is
step function of *θ(l)* for *each class job l*.



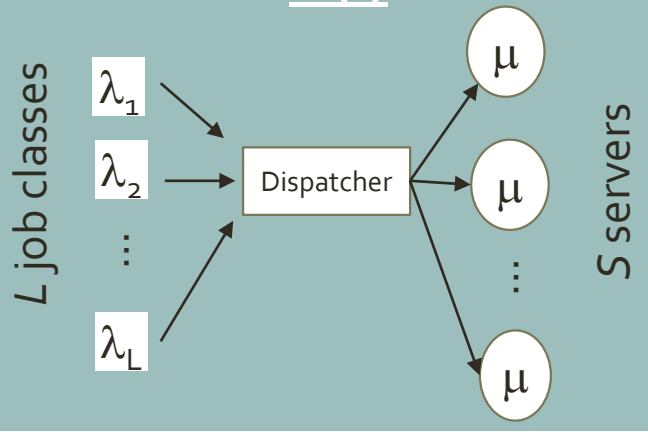
x(l): No. of jobs of class *l* being served

State x	Prob. $p^\pi(\mathbf{x})$	Expected cost $c(\mathbf{x})$	$p^\pi(\mathbf{x})c(\mathbf{x})$	% Exp. cost $\mathbb{E}^\pi(\mathcal{R})$
$[4,0]$	2.2×10^{-4}	0.333×10^3	0.1889	20.0%
$[4,1]$	2.3×10^{-5}	0.333×10^3	0.0075	2.0%
$[4,2]$	1.1×10^{-6}	167×10^3	0.0754	49.7%
$[3,3]$	5.0×10^{-7}	167×10^3	0.0839	22.0%
$[2,4]$	1.3×10^{-7}	167×10^3	0.0210	5.5%
$[1,5]$	1.7×10^{-8}	167×10^3	0.0028	0.74%
$[0,6]$	9.3×10^{-10}	167×10^3	0.0002	0.04%
Total			7.214	100.0%

Table 1: Contribution to the expected cost $\mathbb{E}^\pi(\mathcal{R})$ by each possible blocking state in the loss network system according to expression (10), with $S = 6$, $K = [4,6]$, $\lambda = [1,5]$, $\rho = [0.3, 0.1]$, $R = [2\times10^3, 2\times10^5]$, sorted by decreasing probability $p^\pi(\mathbf{x})$. The bottom four states with occurrence probability smaller than 10^{-6} contribute to about 28% of the expected cost.

Results (2): **Loss Network**

Objective: find optimal capacities $K(l)$



M/M/L/S system

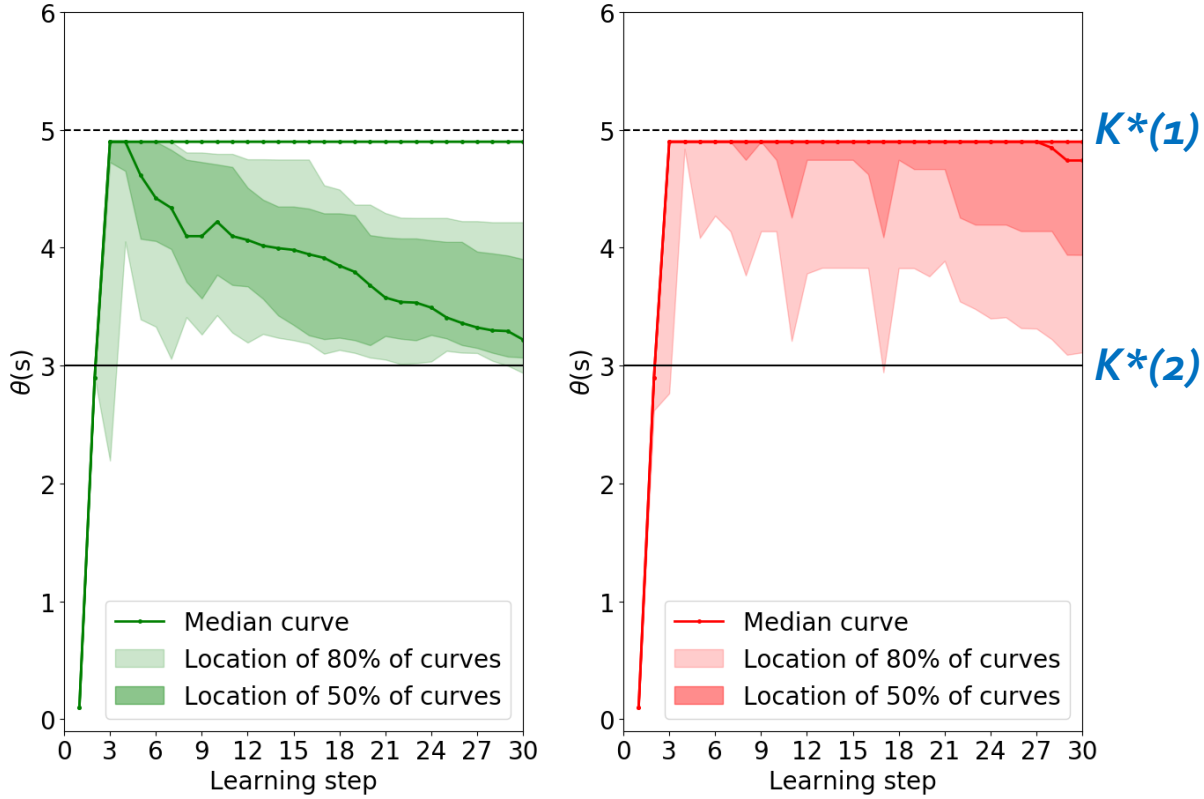
L = 2 job classes
S = 6 servers

Rejection cost depends on the job class: $R(l)$

One policy $\pi(l)$ for *each* job class

Long-run expected reward: $E^{\pi}(R)$

(20 replications)



1) FVRL
2) Monte-Carlo

FVRL reaches the optimum K^* much faster than Monte-Carlo

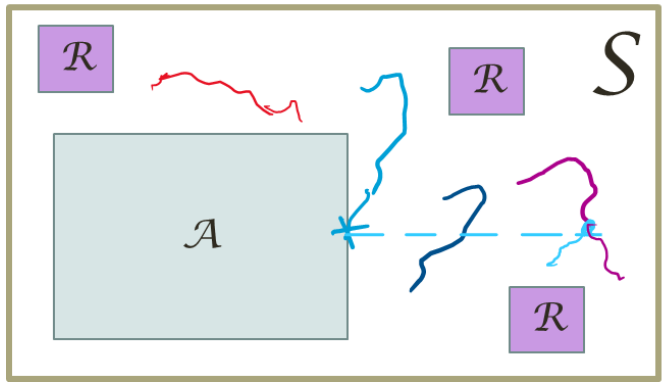
Assumptions and limitations

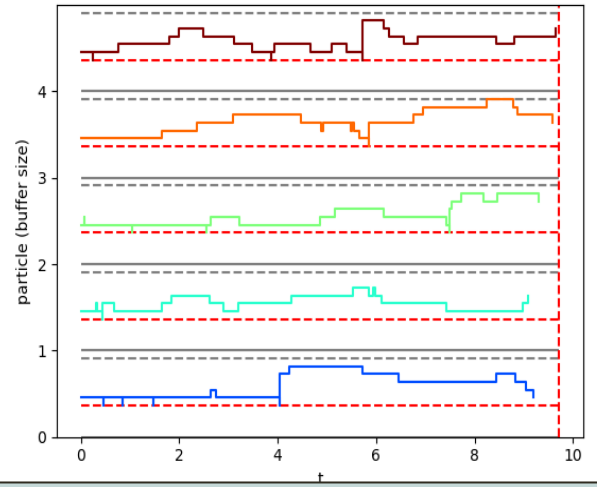
- **Optimisation problem:**
  The objective function must be written as a **long-run expectation** of a measure of interest. (*e.g. long-run expected reward or cost*)
- **System / Environment characteristics:**
  - A **"black-box" simulator** or emulator is needed.
    (*e.g.it is NOT possible to apply the method on previously collected data*)
  - **Finite number of states** (for convergence guarantees).

References:

- **Paper for FVRL on stochastic networks**: https://link.springer.com/article/10.1007/s11134-025-09950-5, "*Fast-exploring reinforcement learning with applications to stochastic networks*", Queueing Systems, Vol. 109 (2025) (open access)
- **Fleming-Viot particle systems**: A. Asselah, P.A. Ferrari, and P. Groisman. "*Quasi-stationary distributions and Fleming-Viot processes in finite spaces*". J. Appl. Probab., 48(2):322–332, **2011**

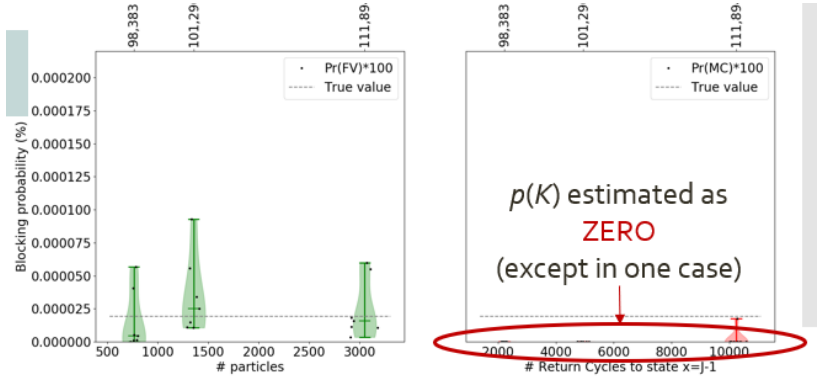
Fleming-Viot METHODOLOGY



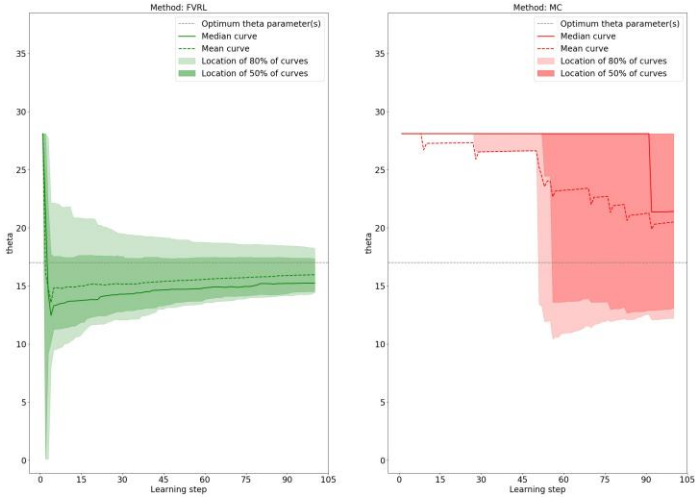


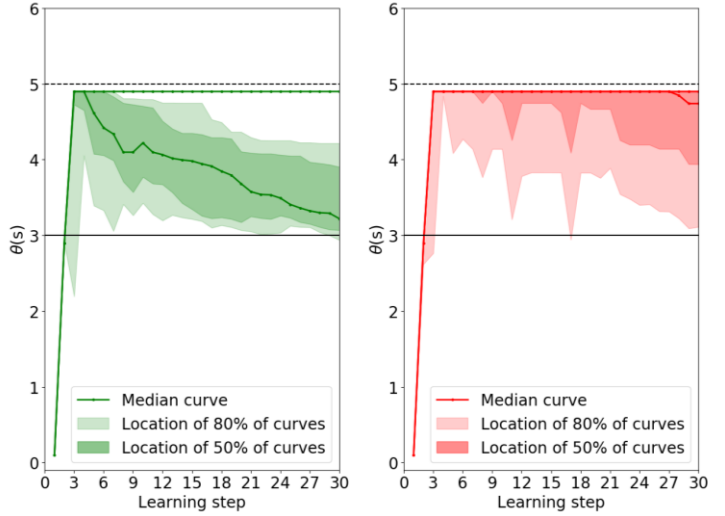
$$p^{\pi}(x) = \frac{\int_0^{\infty} P_J^{\pi}(T_{abs} > t) P_J^{\pi}(X(t) = x | T_{abs} > t)\, dt}{\text{Expected return time to } J\text{–}1 \text{ under } \pi}$$

Fleming-Viot ESTIMATION



Fleming-Viot CONTROL





Questions?

Daniel.Mastropietro@gmail.com

Univ. Toulouse (IRIT), France

Main reference: https://link.springer.com/article/10.1007/s11134-025-09950-5